```
Success, Resume1MICPayload = Crypto_AEAD_DecryptVerify(
    K = S1RK,
    C = Msg1.sigma1ResumeMIC,
    A = Resume1MIC_A,
    N = Resume1MIC_Nonce
)
```

4. If the value of Success is FALSE, the responder SHALL continue processing Sigma1 as if it didn't include any resumption information by continuing the steps in Section 4.14.2.3.5, "Validate Sigma1 Destination ID".
5. If the value of Success is TRUE, the responder SHALL:
   a. Set the Peer Session Identifier in the in the Session Context to the value Msg1.initiatorSessionId.
   b. Send a Sigma2_Resume message.

Generate and Send Sigma2_Resume

The responder SHALL encode and send a Sigma2_Resume message in response to a valid Sigma1 with response.

```
sigma-2-resume-struct => STRUCTURE [ tag-order ]
{
  resumptionID           [1] : OCTET STRING [ length 16 ],
  sigma2ResumeMIC        [2] : OCTET STRING [ length 16 ],
  responderSessionID     [3] : UNSIGNED INTEGER [ range 16-bits ],
  responderSessionParams [4, optional] : session-parameter-struct
}
```

1. The responder SHALL generate a new resumption ID ResumptionID = Crypto_DRBG(len = 128).
2. The responder SHALL generate a session identifier (ResponderSessionId) for subsequent identification of this session. The ResponderSessionId field SHALL NOT overlap with any other existing PASE or CASE session identifier in use by the responder. See Section 4.13.2.4, "Choosing Secure Unicast Session Identifiers" for more details. The responder SHALL set the Local Session Identifier in the Session Context to the value ResponderSessionId.
3. The responder SHALL generate the S2RK key.
4. The responder SHALL generate a resumption MIC:

```
byte Resume2MIC_P[] = {}
byte Resume2MIC_A[] = {}
byte Resume2MIC_Nonce[13] = /* "NCASE_SigmaS2" */
       {0x4e, 0x43, 0x41, 0x53, 0x45, 0x5f, 0x53, 0x69,
```